Hermes Multi-Profile A2A + ADK Runbook v1.0

Status: Field-proven on Zorin OS - 2026-08-26

A reproducible runbook for one Hermes installation with isolated specialist profiles, unique A2A endpoints, and Google ADK RemoteA2aAgent connections.

1. Proven Topology

```
One Hermes installation

default profile          -> A2A :9900
architect_agent          -> A2A :9901
designer_agent           -> A2A :9902
devops_agent             -> A2A :9903

Google ADK Web
  -> hermes_agent
  -> architect_agent
  -> designer_agent
  -> devops_agent
```

Each specialist is a Hermes **profile**, not another Hermes installation. Each profile has isolated configuration, identity (`SOUL.md`), sessions/memory/state, and can be given its own skills and model configuration.

The existing `default` profile is retained as the known-good control agent.

2. Important Hermes Paths

Hermes runtime home:

```
~/.hermes
```

Hermes source checkout in this installation:

```
~/.hermes/hermes-agent
```

Specialist profiles:

```
~/.hermes/profiles/architect_agent
~/.hermes/profiles/designer_agent
~/.hermes/profiles/devops_agent
```

Open the Hermes runtime home in VS Code:

```
code ~/.hermes
```

3. Confirm the Existing Base Profile

```
hermes profile list
```

Field-proven baseline:

```
default    deepseek-v4-flash:0731-cloud
```

The default profile was already working through A2A on port `9900` and ADK Web before specialist profiles were created.

4. Create the Specialist Profiles

The working command is:

```
hermes profile create architect_agent --clone
hermes profile create designer_agent --clone
hermes profile create devops_agent --clone
```

Important correction discovered during the run: do **not** use:

```
hermes profile create architect_agent --clone default
```

On the tested Hermes build this fails with `unrecognized arguments: default`.

`--clone` by itself clones the current/default profile.

Verify:

```
hermes profile list
```

Expected profiles:

```
default
architect_agent
designer_agent
devops_agent
```

5. Configure Agent Identities

Edit each profile's `SOUL.md`.

Architect

File:

~/.hermes/profiles/architect_agent/SOUL.md

```
# Identity

You are the Software Architect for a small AI-powered software team.

Your job is to:
- turn messy ideas into clear technical plans
- define architecture, boundaries, interfaces, and risks
- protect scope and simplicity
- prefer modular, testable, cloud-ready designs
- avoid coding unless explicitly asked
- explain tradeoffs before recommending complexity

When asked who you are, clearly identify yourself as the Software Architect.

Your tone is calm, practical, direct, and technical.
```

Designer

File:

~/.hermes/profiles/designer_agent/SOUL.md

```
# Identity

You are the Product and UI Designer for a small AI-powered software team.

Your job is to:
- turn product requirements into clear user experiences
- define screens, flows, layouts, and interaction behavior
- protect consistency across the interface
- prefer simple, usable designs over decorative complexity
- surface unclear product decisions before designing around them
- avoid backend architecture and implementation unless explicitly asked

When asked who you are, clearly identify yourself as the Product and UI Designer.

Your tone is visual, practical, user-focused, and concise.
```

DevOps

File:

~/.hermes/profiles/devops_agent/SOUL.md

```
# Identity

You are the DevOps Architect for a small AI-powered software team.

Your job is to:
- design simple, reliable deployment architectures
- prefer containerized and cloud-ready solutions
- plan CI/CD, environment configuration, secrets, DNS, and observability
- minimize operational complexity
- separate application concerns from infrastructure concerns
- identify deployment risks before production
- avoid application feature design unless explicitly asked

When asked who you are, clearly identify yourself as the DevOps Architect.
```

```
Your tone is operational, cautious, practical, and concise.
```

6. Configure A2A Per Profile

The cloned `.env` initially contained:

```
A2A_AGENT_NAME="ADK Hermes"
A2A_PEER_TOKENS=adk:<TOKEN>
A2A_PORT=9900
```

Keep the peer-token configuration unchanged for this prototype, but give each specialist a unique name and port.

Architect `.env`

```
A2A_AGENT_NAME="Architect Agent"
A2A_PEER_TOKENS=adk:<TOKEN>
A2A_PORT=9901
```

Designer `.env`

```
A2A_AGENT_NAME="Designer Agent"
A2A_PEER_TOKENS=adk:<TOKEN>
A2A_PORT=9902
```

DevOps `.env`

```
A2A_AGENT_NAME="DevOps Agent"
A2A_PEER_TOKENS=adk:<TOKEN>
A2A_PORT=9903
```

Security: never commit or publish real peer tokens. Use placeholders in documentation and `.env.example` files.

7. Start the Hermes Gateways

Keep the original/default gateway on `9900` if already running.

Start each specialist in its own terminal/process:

```
hermes --profile architect_agent gateway run
```

```
hermes --profile designer_agent gateway run
```

```
hermes --profile devops_agent gateway run
```

A warning about messaging-platform user allowlists was observed during startup. It did not prevent the tested A2A connection.

8. Verify the A2A Agent Cards

Architect:

```
curl http://127.0.0.1:9901/.well-known/agent-card.json
```

Designer:

```
curl http://127.0.0.1:9902/.well-known/agent-card.json
```

DevOps:

```
curl http://127.0.0.1:9903/.well-known/agent-card.json
```

The Architect card was field-verified to return:

```
"name": "Architect Agent"
"url": "http://127.0.0.1:9901/"
```

plus A2A capabilities, skills, JSON-RPC interface information, and bearer security.

9. Connect Each Profile to Google ADK

The already-working ADK remote-agent module was copied for each specialist.

Representative Architect configuration:

```
import httpx

from google.adk.agents.remote_a2a_agent import RemoteA2aAgent

root_agent = RemoteA2aAgent(
    name="architect_agent",
    description="Software Architect connected through Hermes A2A.",
    agent_card="http://127.0.0.1:9901/.well-known/agent-card.json",
    httpx_client=httpx.AsyncClient(
        headers={
            "Authorization": "Bearer <TOKEN>",
        }
    ),
)
```

Use the corresponding Agent Card for each agent:

```
default          http://127.0.0.1:9900/.well-known/agent-card.json
architect_agent  http://127.0.0.1:9901/.well-known/agent-card.json
designer_agent   http://127.0.0.1:9902/.well-known/agent-card.json
devops_agent     http://127.0.0.1:9903/.well-known/agent-card.json
```

The ADK-side pattern is therefore repeatable: copy the known-good `RemoteA2aAgent` package, rename the ADK agent, and change the Agent Card endpoint.

10. Acceptance Test

Start ADK Web using the existing project procedure.

Select each agent and ask:

```
Who are you and what is your job?
```

Expected:

• `architect_agent` identifies as Software Architect.

• `designer_agent` identifies as Product/UI Designer.

• `devops_agent` identifies as DevOps Architect.

• `default` remains the original base Hermes agent.

Field result: all three specialist profiles successfully responded through ADK Web with their configured identities.

This proves:

```
ADK Web
 -> ADK RemoteA2aAgent
 -> unique A2A port
 -> Hermes profile
 -> profile SOUL.md
 -> specialist response
```

11. Adding Specialist Document Stacks Later

This was intentionally deferred during the plumbing proof.

Each profile can receive specialist skills/reference material under its own profile area. A useful shape is:

```
~/.hermes/profiles/architect_agent/
■■■ skills/
    ■■■ architecture/
        ■■■ SKILL.md
        ■■■ references/
            ■■■ APP_ARCHITECTURE_MANUAL.md
            ■■■ DATABASE_MANUAL.md
            ■■■ AUTH_MANUAL.md
            ■■■ API_AND_SERVICES_MANUAL.md
```

First prove identity and transport; then add knowledge stacks as a separate testable change.

12. Preserving the Hermes Setup

The critical runtime state is under:

```
~/.hermes
```

For a practical workstation backup, stop Hermes gateways first so files are not changing during capture, then create a protected archive:

```
mkdir -p ~/backups
tar \
  --exclude="$HOME/.hermes/cache" \
  --exclude="$HOME/.hermes/audio_cache" \
  --exclude="$HOME/.hermes/image_cache" \
  -czf "$HOME/backups/hermes-backup-$(date +%Y%m%d-%H%M%S).tar.gz" \
  -C "$HOME" .hermes
```

This intentionally preserves the Hermes runtime home, including profiles, identities, configuration, skills, sessions/memories/state and the local source checkout present in this installation.

Because `.env` files may contain API keys and A2A credentials:

• treat the archive as a **secret-bearing backup**;

• do not commit it to Git;

• do not place it in a public bucket;

• encrypt it before storing it outside a trusted machine.

A second, cleaner disaster-recovery strategy should eventually separate:

1. **Version-controlled configuration/doctrine** - profile templates, sanitized configs, SOULs, skills, scripts.
2. **Secrets** - secret manager or encrypted backup.
3. **Mutable runtime data** - memories, sessions, state.
4. **Hermes source/version pin** - Git commit/tag or container image version.

For the current prototype, a full `~/.hermes` backup is the simplest safe preservation point.

Restore concept

On a replacement machine with the same user ownership expectations:

```
tar -xzf hermes-backup-YYYYMMDD-HHMMSS.tar.gz -C "$HOME"
```

Then verify permissions, activate/install the required Hermes environment/version, run:

```
hermes profile list
```

and test each gateway/Agent Card before reconnecting ADK.

Do not treat restore as proven until a restore drill has actually been performed.

13. Final Proven Result

```
ONE HERMES INSTALLATION
        |
        +-- default           :9900
        +-- architect_agent   :9901
        +-- designer_agent    :9902
        +-- devops_agent      :9903
                    |
                    | A2A
                    v
               GOOGLE ADK
                    |
                    v
                 ADK WEB
```

This is the proven foundation for a repeatable multi-profile Hermes workforce. Model specialization, specialist document stacks, Docker packaging, cloud deployment, and ADK orchestration should be added as separate phases after this baseline remains reproducible.